esercitazione-3-canale1-template

Private template

Watch 0 Fork 0 Use this template

About

No description, website, or topics provided.

Readme

Activity Custom properties

0 stars 0 watching 0 forks

1 Branch 0 Tags Go to file T Go to file Code

lorenzo-rovigatti Aggiorna la scheda 07570c8 · 19 hours ago

.gitignore	Aggiorna il testo	5 days ago
README.md	Aggiorna la scheda	19 hours ago
autograde_part1.py	Piccolo cambio alla scheda e ...	2 days ago
autograde_part2.py	Piccolo cambio alla scheda e ...	2 days ago
scheda.pdf	Aggiorna la scheda	19 hours ago

README

Esercitazione 3 - Il metodo velocity Verlet: oscillatore armonico e pendolo semplice

Obiettivi

In questa esercitazione introdurremo il metodo **velocity Verlet** e ne confronteremo

l'accuratezza con i metodi di Eulero ed Eulero-Cromer studiati nell'esercitazione precedente. Impareremo a stimare empiricamente l'ordine di convergenza di un integratore. Applicheremo poi velocity Verlet al pendolo semplice, confrontando il regime di piccole oscillazioni con quello non lineare.

Prima di cominciare fate login sul sito di GitHub.com con le vostre credenziali, e poi eseguite il comando `github-inizio.sh` (vedi esercitazione 0) dal terminale che volete usare. Create una cartella (ad esempio `LFC_XX`, dove `XX` è il nome del vostro gruppo), entrate al suo interno e fate il `git clone` del repository creato al momento dell'accettazione dell'esercitazione. Per assicurarvi che i *commit* siano a vostro nome, una volta entrati nella cartella clonata date i seguenti comandi:

```
git config user.name "Nome Cognome"
git config user.email "email-associata-a-GitHub"
```

Consegna dell'esercitazione

La consegna deve contenere:

- `oscillatore.c`, con i tre metodi di integrazione;
- `pendolo.c`;
- `analisi.md`, con le risposte alle domande di entrambe le parti;
- i grafici a supporto dell'analisi;
- gli eventuali script usati per generare i dati e i grafici.

La consegna richiede un tag (vedi esercitazione 0): dopo il push che corrisponde al commit che volete consegnare, eseguite

```
submission_tag="submit/$(date -u +%Y%m%dT%H%M%SZ)"
git tag "$submission_tag"
git push origin "$submission_tag"
```

Il tag include solo modifiche già registrate in un commit. Per una correzione, fate un nuovo commit, push e un nuovo tag.

1. Velocity Verlet applicato all'oscillatore armonico

Il sistema

Consideriamo l'oscillatore armonico utilizzato nell'esercitazione 2, per cui conosciamo la soluzione esatta:

$$x_{\text{esatto}}(t) = x_0 \cos(\omega_0 t) + \frac{v_0}{\omega_0} \sin(\omega_0 t)\,.$$

Definiamo l'errore globale sulla posizione al tempo finale t_f come

$$\epsilon_G(\Delta t) = \frac{|x(t_f) - x_{\text{esatto}}(t_f)|}{A},$$

dove $A = \sqrt{x_0^2 + (v_0 / \omega_0)^2}$ è l'ampiezza dell'oscillazione.

Il metodo

Il metodo di velocity Verlet si può riassumere con questi quattro passi:

1. Aggiornamento della velocità, prima fase: $v_{n+1/2} = v_n + \frac{1}{2} a_n \Delta t$.
2. Aggiornamento della posizione: $x_{n+1} = x_n + v_{n+1/2} \Delta t = x(t) + v_n \Delta t + \frac{1}{2} a_n \Delta t^2$.
3. Calcolo della forza (e quindi dell'accelerazione) utilizzando la nuova posizione: $x_{n+1} \rightarrow a_{n+1} = F_{n+1} / m$.
4. Aggiornamento della velocità, seconda fase: $v_{n+1} = v_{n+1/2} + \frac{1}{2} a_{n+1} \Delta t = v_n + \frac{1}{2}(a_n + a_{n+1})\Delta t$.

Analisi e consegne

Rispondete alle seguenti domande nella prima parte del file `analisi.md` :

1.1. Quale dipendenza dell'errore ϵ_G da Δt osservate per Eulero, Eulero-Cromer e velocity-Verlet? Nei tre diversi casi, qual è la pendenza dei dati in un grafico log-log?

1.2. Nella scorsa esercitazione avete imparato che l'algoritmo di Eulero-Cromer funziona meglio dell'algoritmo di Eulero. Confronta l'algoritmo di Eulero-Cromer e quello di velocity Verlet. Quale algoritmo è più indicato per integrare l'oscillatore armonico?

Supportate le vostre considerazioni con almeno il grafico log-log di ϵ_G per i tre metodi, da allegare alla consegna. Considerate anche lo studio di energia e spazio delle fasi per rispondere alla seconda domanda.

Estensioni opzionali

Si consiglia di passare al punto 2 (il pendolo semplice) dopo non più di un'ora e mezzo dall'inizio ell'esercitazione, anche se i punti opzionali non sono stati completati.

1.3 **Stabilità.** Fissate ω_0 e aumentate progressivamente Δt: esiste un valore oltre il quale la soluzione numerica ottenuta con velocity Verlet diverge? Esprimete la soglia in termini della

grandezza adimensionale $\omega_0 \Delta t$.

1.4 **Ordine di convergenza dell'energia.** Ripetete lo studio dell'ordine di convergenza usando come misura dell'errore la massima deviazione relativa dell'energia, $\max_t |E(t) - E(0)| / E(0)$.

Implementazione del codice

Estendete il programma `oscillatore.c` sviluppato nell'esercitazione precedente aggiungendo il metodo `velocity-verlet` e modificandolo in modo che in output stampi solo Δt e ϵ_G e contenga un ciclo su Δt. Dato che Δt diventa una variabile e non è più necessario stampare informazioni intermedie, l'interfaccia diventa

```
./oscillatore metodo nsteps x0 v0 omega0
```

e i valori ammessi per `metodo` diventano `euler` , `euler-cromer` e `velocity-verlet` (oppure 1, 2 e 3 se avete scelto le etichette numeriche).

Consigli:

1. Potete usare i seguenti valori di Δt: 0.001, 0.002, 0.004, 0.008, 0.016, 0.032, 0.064.
2. Prima di aggiungere il ciclo su Δt vi consigliamo di lasciare interfaccia e output dell'esercitazione 1 per assicurarvi che l'integrazione con velocity Verlet funzioni. Una volta assicurati che il codice funziona, copiate il file e usatelo come base per la seconda parte!
3. Quando estenderete il codice aggiungendo il ciclo sui Δt, utilizzate `nsteps` per calcolare il *tempo di simulazione* t_f utilizzando il Δt più piccolo. Per ogni altro valore di Δt , i passi da simulare diventano quindi $t_f / \Delta t$.

Generazione dell'output

A differenza di quanto fatto nelle esercitazioni precedenti, l'output finale del codice dovrebbe contenere due colonne, Δt ed ϵ_G, per diversi valori di Δt.

Come rispondere alle domande

Fissate le condizioni iniziali, ω_0 e il tempo finale t_f, e fate variare Δt; per mantenere t_f costante dovete scegliere `nsteps` $= t_f / \Delta t$;

Come abbiamo visto a lezione, ci aspettiamo $\epsilon_G \propto \Delta t^p$, e quindi $\log \epsilon_G = p \log \Delta t +$ cost . : in un grafico log-log i dati si dispongono su una retta la cui pendenza è l'ordine di convergenza p. Se sapete come farlo, potete stimare p con un fit lineare dei logaritmi, oppure confrontate più semplicemente i dati con rette di pendenza nota tracciate sullo stesso grafico.

2. Il pendolo semplice

Il sistema

L'importanza degli algoritmi numerici risiede nella possibilità di studiare problemi che non possono essere studiati analiticamente in termini di funzioni semplici, come ad esempio il problema del pendolo. In questi casi, procedere numericamente è spesso l'unica soluzione.

Consideriamo ora un pendolo semplice di lunghezza L e massa m, descritto in forma di sistema del primo ordine come

$$\begin{cases} \frac{d\theta}{dt} = \omega \\ \frac{d\omega}{dt} = -\frac{g}{L}\sin\theta, \end{cases}$$

dove $\omega(t) = \frac{d\theta}{dt}$ è la velocità angolare. Consideriamo come condizioni iniziali l'angolo iniziale $theta(0) = theta_0$ rispetto alla verticale e la velocità angolare iniziale $\omega(0) = 0$ (da non confondere con la pulsazione ω_0 dell'oscillatore armonico della prima parte). Questa seconda scelta è fatta per comodità, ma potete divertirvi a pensare a cosa succederebbe scegliendo $\omega(0) \neq 0$.

L'energia meccanica è

$$E = \frac{1}{2} m L^2 \omega^2 + mgL(1 - \cos\theta)\,.$$

Nel limite delle piccole oscillazioni, $\sin\theta \simeq \theta$, il pendolo si comporta come un oscillatore armonico con pulsazione $\omega_0 = \sqrt{g / L}$ e periodo $T_0 = 2\pi / \omega_0$. Al di fuori di questo limite l'equazione è non lineare e non ammette una soluzione in termini di funzioni elementari: la simulazione numerica diventa quindi lo strumento naturale per studiarla.

Il metodo

Useremo velocity Verlet, con la stessa struttura vista nella prima parte, sostituendo $x \to \theta$, $v \to \omega$ e l'accelerazione con l'accelerazione angolare

$$\alpha(\theta) = -\frac{g}{L}\sin\theta\,.$$

Analisi e consegne

Rispondete alle seguenti domande nella seconda parte del file `analisi.md` :

2.1 Per quali ampiezze iniziali l'approssimazione di piccolo angolo descrive bene il moto? Leggete la sezione **Come rispondere alle domande** per un aiuto!

2.2 Come cambiano il periodo e la traiettoria nello spazio delle fasi aumentando l'ampiezza angolare iniziale θ_0?

2.3 L'energia in funzione del tempo rimane costante durante l'integrazione? Come dipende il risultato dalla scelta di Δt?

Supportate le vostre considerazioni con dei grafici (almeno $\theta(t)$ e lo spazio delle fasi per diverse ampiezze, e il periodo in funzione dell'ampiezza). Come nell'esercitazione precedente, potete inserire i grafici direttamente nel file markdown usando la sintassi `![](grafico.png)`.

Estensione opzionale

2.4 **Moto di rotazione.** Scegliete una velocità angolare iniziale abbastanza grande da far compiere al pendolo giri completi. Come appare la traiettoria nello spazio delle fasi? Qual è la condizione sull'energia che separa oscillazioni e rotazioni? (Per graficare può essere utile riportare θ nell'intervallo $[-\pi, \pi)$.)

2.5 **Animazione.** Realizzate un video che mostri il moto del pendolo affiancato alla sua traiettoria nello spazio delle fasi.

Implementazione del codice

Scrivete un programma C, il cui file sorgente **deve chiamarsi** `pendolo.c`, che integri la dinamica mediante **velocity Verlet**. Usare il codice scritto per il punto (o l'esercitazione) precedente come base potrebbe farvi risparmiare tempo! Il programma deve essere eseguito con la sintassi

```
./pendolo dt nsteps stride theta0 omega0 length
```

dove, nell'ordine:

- `dt` è il passo di integrazione Δt;
- `nsteps` è il numero di passi di integrazione;
- `stride` è il numero di passi di integrazione tra due righe consecutive di output;
- `theta0` è l'angolo iniziale $\theta_0 = \theta(0)$, in radianti;
- `omega0` è la velocità angolare iniziale $\omega(0)$, in radianti al secondo;
- `length` è la lunghezza L del pendolo.

Il programma deve verificare che sia stato fornito il numero corretto di argomenti. Fissate

m = 1 e definite l'accelerazione di gravità g = 9.81 m / s^2 come costante nel codice (ad esempio con `#define` o con una variabile `const double`).

Per sin e cos servono `#include <math.h>` e, in compilazione, l'opzione `-lm`.

Generazione dell'output

Valgono le stesse convenzioni della scorsa esercitazione: va stampata sempre la condizione iniziale (t = 0) e poi una riga ogni `stride` passi. Ogni riga deve contenere esattamente quattro colonne separate da spazi, nell'ordine

```
t theta omega E
```

senza intestazioni o altre informazioni. Per quanto riguarda il periodo (vedi sotto), potete stimarlo all'interno del codice di simulazione e stamparlo in un file a parte, oppure creare un altro programma in C (o, se preferite, in Python) che analizzi la traiettoria stampata dal codice di simulazione.

Come rispondere alle domande

Scelta dei parametri. Il passo di integrazione va scelto in relazione alla scala di tempo del sistema, cioè al periodo T_0: un Δt pari a qualche centesimo o millesimo di T_0 è un buon punto di partenza. Integrate per un numero di periodi sufficiente a stimare il periodo con precisione. Per esplorare i diversi regimi conviene partire da fermo ($\omega(0) = 0$) e variare θ_0 da valori piccoli (pochi gradi) fino a valori vicini a π.

Stima del periodo. È più semplice stimare il periodo come intervallo tra zeri di $\theta(t)$ che tra massimi o minimi, ma bisogna scegliere gli zeri in modo opportuno:

- due zeri consecutivi sono separati da *mezzo* periodo;
- poiché i dati sono discreti, uno zero cade in generale tra due righe dell'output: lo individuate cercando un cambio di segno tra righe consecutive, e potete stimarne la posizione con un'interpolazione lineare tra i due punti;
- mediando su più periodi si ottiene una stima più precisa. Per questa analisi conviene usare `stride` pari a 1, oppure piccolo, ma state attenti a non fare simulazioni troppo lunghe!

Piccole e grandi oscillazioni. Per decidere quando l'approssimazione di piccolo angolo "descriva bene" il moto serve un criterio quantitativo: ad esempio, potete richiedere che la differenza relativa $|T - T_0| / T_0$ sia inferiore all'1%. Un grafico di T / T_0 in funzione di θ_0 riassume bene i risultati. Confrontate inoltre, sugli stessi grafici, $\theta(t)$ e le traiettorie nello spazio delle fasi (θ, ω) ottenute per ampiezze diverse.

Energia. Non avendo a disposizione una soluzione esatta, la conservazione dell'energia è il principale controllo sulla qualità dell'integrazione. Graficate $E(t)$, o meglio la variazione relativa $|E(t) - E(0)| \ / \ E(0)$, per diversi valori di Δt.

Releases

No releases published

Packages

No packages published

Contributors 2



lorenzo-rovigatti Lorenzo Rovigatti

carlomt Carlo Mancini

Languages

Python 100%